

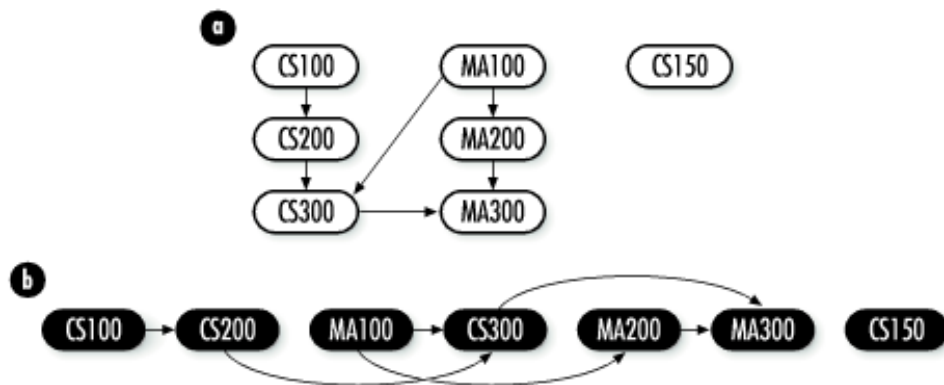
CSE 221: Algorithms

Chapter 7 — Graph Algorithms

Prepared by Ariyan Hossain [AYO]

1 Topological Sort

A **topological ordering** of a graph is a linear ordering of all vertices such that for every directed edge (u, v) , vertex u appears before v in the ordering.



Topological ordering of a sequence of courses

Conditions for Topological Sort

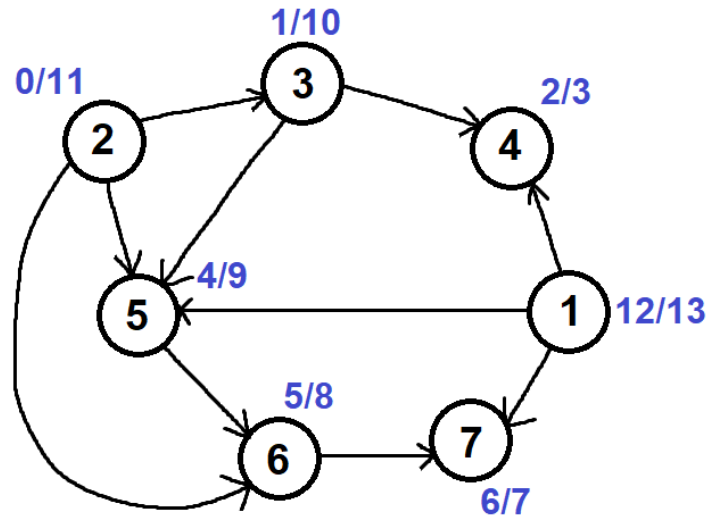
To find a topological ordering, we need a graph that is:

- **Directed:** An undirected edge between u and v implies both $u \rightarrow v$ and $v \rightarrow u$, so each depends on the other and no valid topological order exists.
- **Acyclic (no cycles):** A cycle will always create a contradiction because all the vertices in a cycle are indirectly dependent on each other.

1.1 Simulation (Topological Sort)

A standard way to find a topological order is to run **DFS** and then list vertices in **decreasing order of finishing time**.

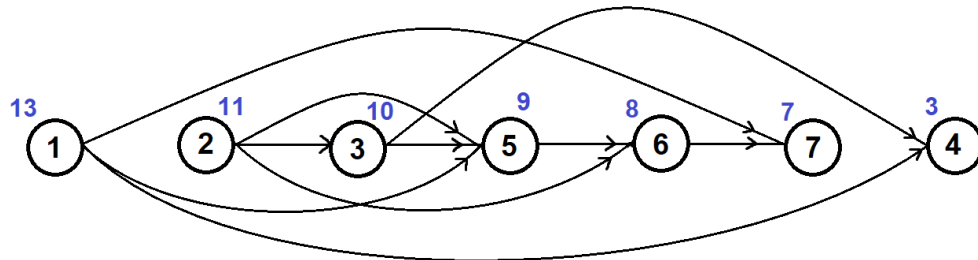
DFS can start from any source vertex; in the following example, we start from vertex 2.



DFS on the graph with source vertex = 2

If the finishing times are arranged in a decreasing manner we will get the ordering of the vertices as shown below.

End Time	13	11	10	9	8	7	3
Vertices	1	2	3	5	6	7	4



Topological ordering of the graph based on DFS finishing times

1.2 Time Complexity (Topological Sorting)

- Each traversal visits every node once $\Rightarrow \mathcal{O}(V)$
- Each traversal traverses every edge once $\Rightarrow \mathcal{O}(E)$
- Processing vertices in reverse finishing-time order using a stack does not change the complexity, since pushing and popping all vertices $\Rightarrow \mathcal{O}(V)$

Therefore, the total running time remains

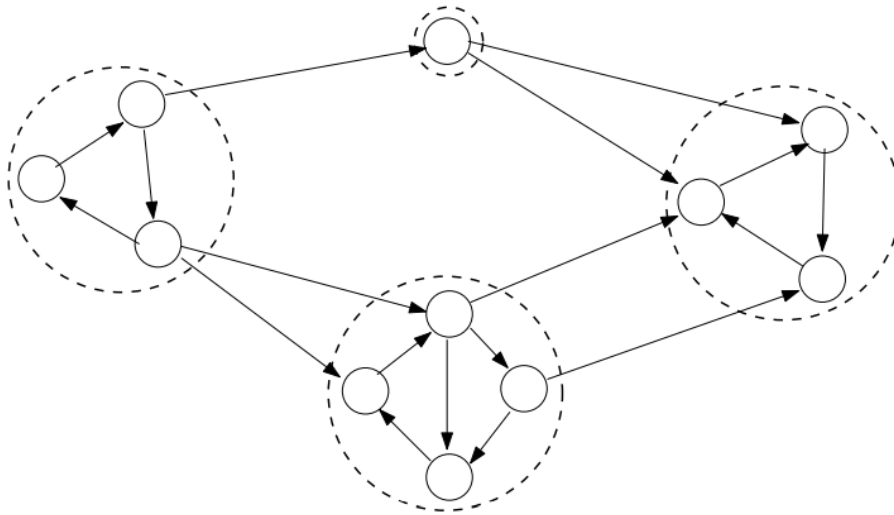
$$\mathcal{O}(V) + \mathcal{O}(E) + \mathcal{O}(V) = \mathcal{O}(V + E)$$

2 Strongly Connected Components (SCC)

In a directed graph $G = (V, E)$, two nodes u and v are strongly connected if and only if there is a path from u to v and a path from v to u .

A **Strongly Connected Component (SCC)** is a **maximal group of vertices where each vertex can reach every other vertex in that group**.

An undirected graph is always strongly connected but a directed graph is not always strongly connected.

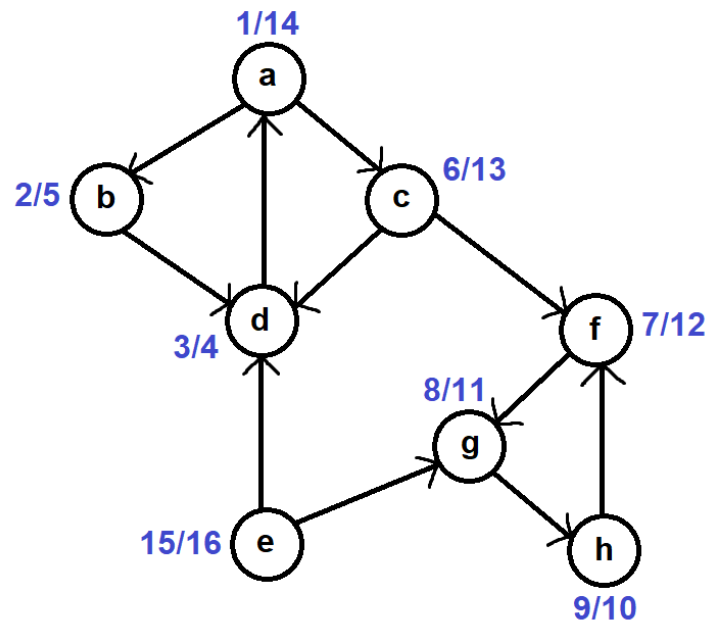


Example of strongly connected components.

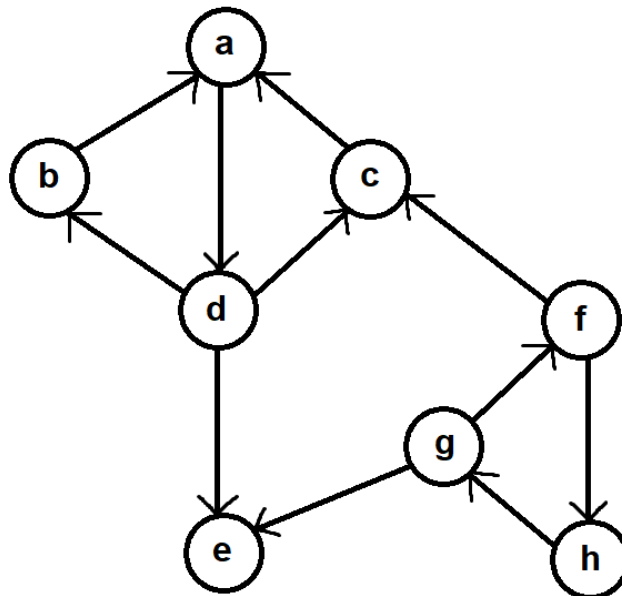
2.1 Kosaraju's Algorithm

1. Run DFS on G .
2. Reverse all edges to obtain G^T .
3. Run DFS on G^T in decreasing order of finishing time from the first DFS.
4. Each DFS tree forms one SCC.

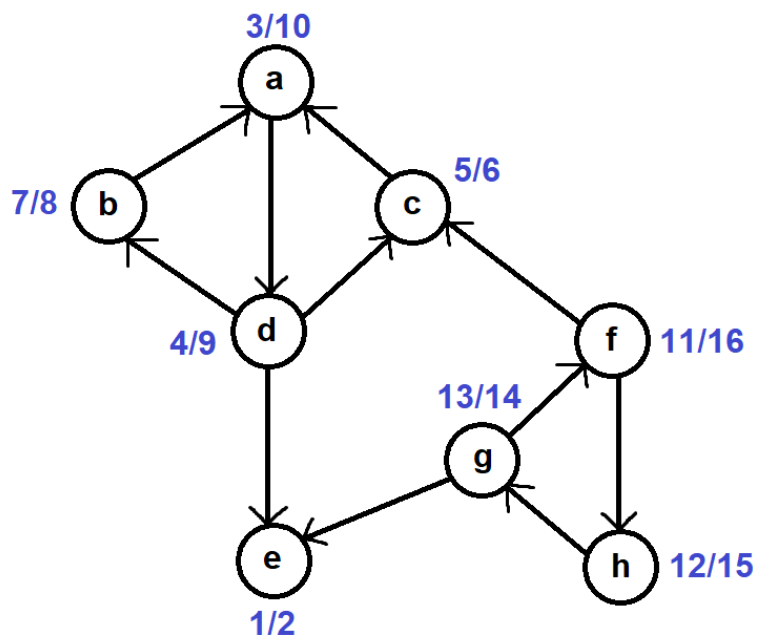
2.2 Simulation (Kosaraju's Algorithm)



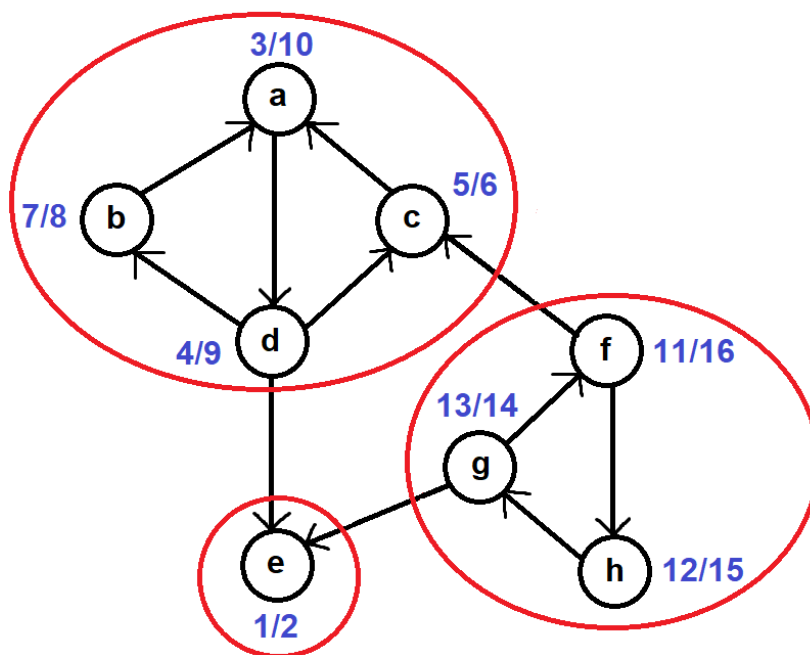
Step 1: Run DFS using any source, let source vertex = a



Step 2: Transpose the graph (Reverse Edges)



Step 3: Run DFS starting from decreasing finishing time from step 1



Step 4: Combine vertices of each DFS tree from step 3

2.3 Time Complexity (Kosaraju's Algorithm)

- First DFS computes finishing times, visiting every node once $\Rightarrow \mathcal{O}(V + E)$
- Constructing the transpose graph reverses every edge once $\Rightarrow \mathcal{O}(V + E)$
- Second DFS explores the transposed graph in reverse finishing-time order, visiting every node once $\Rightarrow \mathcal{O}(V + E)$

- Processing vertices in reverse finishing-time order using a stack does not change the complexity, since pushing and popping all vertices $\Rightarrow \mathcal{O}(V)$

Therefore, the total running time remains

$$\mathcal{O}(V + E) + \mathcal{O}(V + E) + \mathcal{O}(V + E) + \mathcal{O}(V) = \mathcal{O}(V + E)$$

3 Dijkstra's Algorithm

BFS can be used to find the shortest path from a source when the graph is unweighted. The question then arises: how should this problem be approached when the graph is weighted? One possible method is to calculate the weights of all possible paths from the source to every other node, but more optimal solutions exist.

A **greedy algorithm** called Dijkstra's Algorithm is used for this purpose. A greedy algorithm makes the best local choice at each step. It calculates the **single-source shortest path** from a source vertex to every other vertex. Dijkstra's algorithm assumes that all edge weights in the input graph are non-negative.

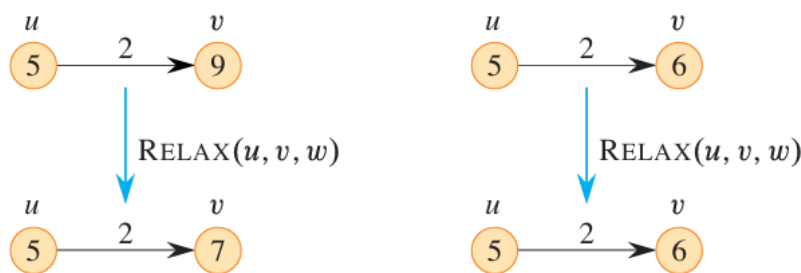
Shortest paths contain no cycles, that is, they are simple paths. Since any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices, it also contains at most $|V| - 1$ edges. Therefore, any shortest path can be assumed to contain at most $|V| - 1$ edges.

Relaxation Operation

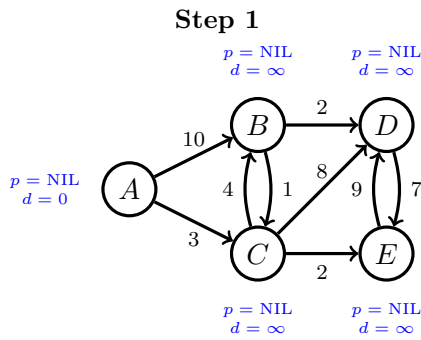
$\text{RELAX}(u, v, w)$: if $d[v] > d[u] + w(u, v)$, then $d[v] \leftarrow d[u] + w(u, v)$, $\pi[v] \leftarrow u$.

The process of relaxing an edge (u, v) consists of testing whether going through vertex u improves the shortest path to vertex v found so far and, if so, updating $d[v]$ and $\pi[v]$.

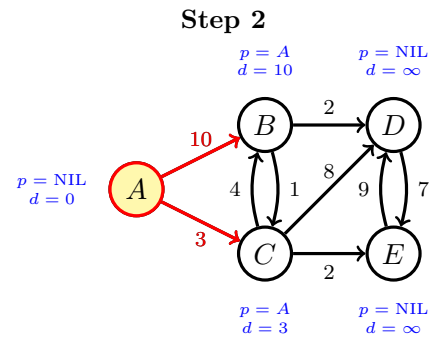
In Dijkstra's algorithm, each edge is relaxed exactly once.



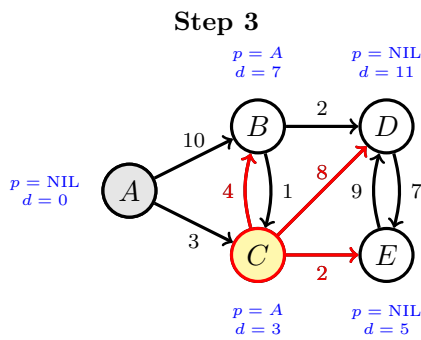
Successful vs Unsuccessful Relaxation

3.1 Simulation (Dijkstra's Algorithm from source A)

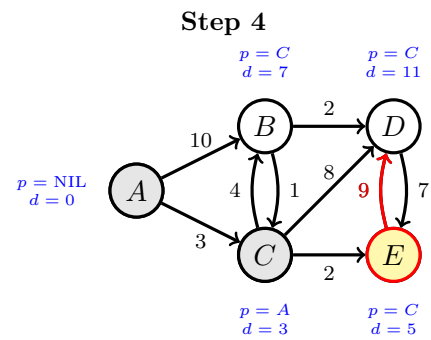
Priority Q: $[A, B, C, D, E]$
Current: none yet
Neighbors: none yet



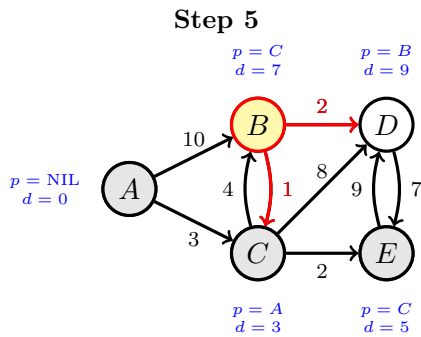
Priority Q: $[B, C, D, E]$
Current: A
Neighbors: B, C



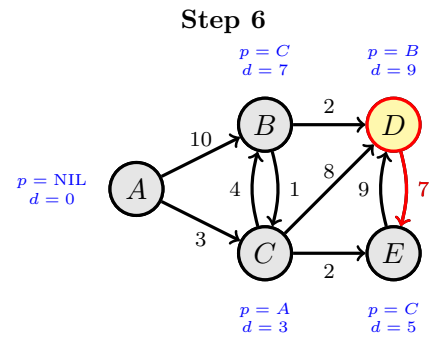
Priority Q: $[B, D, E]$
Current: C
Neighbors: B, D, E



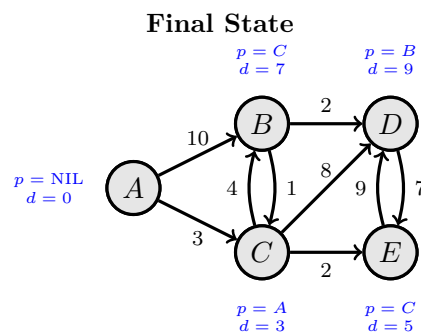
Priority Q: $[B, D]$
Current: E
Neighbors: D



Priority Q: $[D]$
Current: B
Neighbors: C, D



Priority Q: $[\]$
Current: D
Neighbors: E



Priority Q: $[\]$
Current: none
Neighbors: none

3.2 Pseudocode (Dijkstra's Algorithm)

Dijkstra's Algorithm on graph G from source s

```

DIJKSTRA( $G, s$ )
  for each  $v$  in  $V[G]$ 
    do  $d[v] \leftarrow \text{infinity}$ 
        $pi[v] \leftarrow \text{NIL}$ 
   $d[s] \leftarrow 0$ 
   $Q \leftarrow V[G]$ 
  while  $Q$  is not empty
    do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
       for each vertex  $v$  in  $\text{Adj}[u]$ 
         do if  $d[v] > d[u] + w(u, v)$ 
              then  $d[v] \leftarrow d[u] + w(u, v)$ 
                    $pi[v] \leftarrow u$ 

```

3.3 Time complexity of Dijkstra's Algorithm

To perform EXTRACT-MIN in Dijkstra's Algorithm, a **binary min-heap** data structure is used as a priority queue. A min-heap is a tree-based data structure with the property that every node has a smaller key than its children.

- **findMin** takes $\mathcal{O}(1)$ per call and is performed at most V times $\Rightarrow \mathcal{O}(V)$
- **removeMin** takes $\mathcal{O}(\log V)$ per call and is performed at most V times $\Rightarrow \mathcal{O}(V \log V)$
- **updateKey** takes $\mathcal{O}(\log V)$ per call and is performed at most E times $\Rightarrow \mathcal{O}(E \log V)$

Therefore, Dijkstra's algorithm runs in

$$\mathcal{O}(V) + \mathcal{O}(V \log V) + \mathcal{O}(E \log V) = \mathcal{O}((V + E) \log V)$$

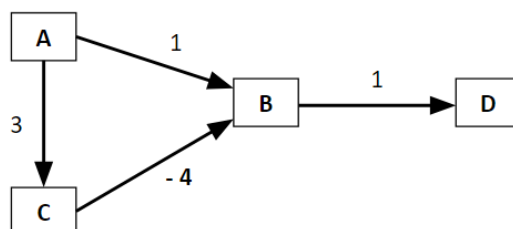
For connected graphs, $E \geq V - 1$, so this is often written as $\mathcal{O}(E \log V)$.

Follow-up Questions

Q) What is the major drawback of Dijkstra's Algorithm?

Ans: Dijkstra's Algorithm does not guarantee to provide the shortest path when the edges are negative since it is a greedy algorithm.

Try out the algorithm on the following graph:



Q) What is the greedy property of Dijkstra's Algorithm?

Ans: Dijkstra's greedy property is: at each step, it permanently chooses the unvisited node with the smallest current tentative distance from the source.